

数据表示与位运算实验报告

杨鑫源 25300680192

一、设计函数思路：

1. signMask

将数值 1 左 31 位，得到仅最高位为 1 的数；再对其执行算术右移 31 位，符号位会填充所有低位，最终得到全 1 的 32 位符号掩码

2. bitXor

通过德摩根定律用与非运算组合实现异或。异或逻辑为“两位不同则为 1”，等价于(A 且非 B) 或 (非 A 且 B)，转换为与非形式就是 $\sim(\sim(A \& \sim B) \& \sim(\sim A \& B))$ ，仅用按位与和按位取反即可完成异或运算

3. negativePart

先将原数算术右移 3 位，得到符号掩码：负数时掩码为全 1，非负数时为全 0。将掩码与原数的补码相反数 ($\sim x + 1$) 按位与，负数返回对应绝对值，非负数返回 0

4. copyByteWithin

先将源字节通过左移对齐到目标字节的位置，再构造 0xFF 左移到目标位置的字节掩码，用掩码清除目标位置原有数值，最后将对齐后的源字节写入目标位置。

5. logicalShift

C 语言有符号数默认是算术右移，高位会补符号位。先执行正常右移，再构造高位清零掩码：将 1 左移 31 位后右移 n 位再左移 1 位，得到高位 n 个 0、低位全 1 的掩码，与右移结果按位与即可得到逻辑右移效果

6. swapNibblePairs

交换每个字节的高 4 位与低 4 位。先构造 4 位宽的基础掩码 0x0F，通过移位扩展到 32 位全范围。用掩码提取所有低 4 位，左移 4 位变为高 4 位；再用移位后的掩码提取所有高 4 位，右移 4 位变为低 4 位，两者按位或完成半字节交换

7. secondLowestZeroBit

找到倒数第二个 0 比特的位置。先对原数按位取反，将问题转化为找第二个 1 的位置。先用 $x \& -x$ 提取最低位的 1，将其异或回取反后的数以清除最低位；再对剩余数值做一次 $x \& -x$ ，提取的结果就是原数倒数第二个 0 位对应的掩码。

8. oddParity

将数值高低 16 位按位异或，压缩为 16 位；再将高低 8 位异或压缩为 8 位，依次迭代到 1 位。最终最低位就是整个 32 位数的奇偶校验结果

9. rotateRightBits

实现 32 位循环右移 n 位。先将数值逻辑右移 n 位，得到右移后的低位部分；再将数值左移 32-n 位，得到绕到高位的部分，两部分按位或完成循环右移。移位前先对 n 取模 32，避免移位位数超过 31 的未定义行为

10. roundEvenPow2

向最近的 2 的 n 次幂倍数舍入，中间值舍入到偶数倍。先构造 2 的 n 次幂的位掩码，提取低于 n 位的小数部分。小数部分大于一半则向上进位；等于一半时判断整数部分最低位，奇数则进位到偶数；小于一半直接截断。最后用掩码对齐到 2 的 n 次幂边界。

11. midpointTowardFirst

计算两数平均值，向第一个数的方向舍入。采用无溢出中点公式 $(x \& y) + ((x \oplus y) \gg 1)$ 计算平均值，避免中间计算溢出。再判断两数差值的奇偶性：如果 x 大于 y 且差值为奇数，则将中点加 1。大小判断通过符号位组合实现，避免减法溢出。

12. isBetweenEitherOrder

判断 x 是否在 a 、 b 之间，不要求 a 、 b 有序。分别验证两种区间情况： x 在 a 和 b 之间，或 x 在 b 和 a 之间，满足其一即返回真。数值大小比较通过符号位异或和减法结果符号位组合实现：同号时看减法结果符号，异号时正数一定大于负数，规避极值减法的溢出问题。

13. mul5Sat

计算 x 乘以 5，溢出时饱和到整数边界。将 x 左移 2 位得到 4 倍值，加上原数得到 5 倍结果。且左移后与原数符号不一致则溢出；加法时两个加数与结果符号不一致则溢出。溢出时根据原数符号，正数饱和到整数最大，负数饱和到整数最小

14. classifyAdd3

判断三个整数相加的最终溢出方向。分两步计算 $x+y$ 、结果 z ，分别记录每次溢出的方向（正溢出/负溢出）。将两次溢出的偏移量累加，总偏移量为正则最终正溢出返回 1，为负则最终负溢出返回 -1，无溢出或溢出相互抵消则返回 0。

15. floatScaleThreeHalves

将单精度浮点数乘以 $3/2$ ，采用银行家舍入。先拆解出符号位、阶码、尾数三部分，NaN 和无穷值直接返回。给尾数补上隐含的最高位 1，乘以 3 后右移 1 位完成除以 2，右移时执行银行家舍入：移出位为 1 且剩余最低位为 1 则进位。之后重新规格化尾数并调整阶码，阶码溢出则返回对应符号的无穷大，最后拼接回浮点数格式。

16. floatRoundEven

将单精度浮点数舍入到最近整数，中间值舍入到偶数。拆解阶码和尾数，阶码 ≥ 23 时已是整数直接返回；阶码 < -1 时绝对值小 0.5，舍入到 0。其余情况给尾数补上隐含 1，右移对应位数得到整数部分和小数部分：小数部分大于一半则进位，等于一半时看整数部分最低位奇偶性，奇数则进位。最后重新规格化，保留原符号拼接回浮点数格式

17. float_i2f

0 直接返 0，负数先取绝对值并记录符号位。查找最高位 1 的位置，计算带偏移量的阶码值。将尾数左移对齐到 23 位，低位多余部分执行银行家舍入。舍入后若尾数进位则调整阶码，最终拼接符号位、阶码、尾数，返回无符号整数形式的浮点数编码

18. bitCount

构造 1 位、2 位、4 位、8 位、16 位宽的分层掩码，逐层将相邻位的计数值相加：先两两相加得到每 2 位的计数，再四四相加得到每 4 位的计数，依次迭代到 32 位，最终数值就是全部 1 的个数。所有掩码均由 8 位以内的小常量通过移位构造，符合常量范围限制

19. bitReverse

采用分层交换策略：先交换高低 16 位，再交换每个 16 位内部的高低 8 位，接着交换每个 8 位内部的高低 4 位，再交换每个 4 位内部的高低 2 位，最后交换

相邻两位。通过掩码提取对应位段，移位后按位或完成交换。掩码采用递推方式构造，将操作数控制在限制范围内。

二、截图

```
yangxinyuan@localhost:~/ICS2026/Lab1$ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 9/18 operators
secondLowestZeroBit: 8/8 operators
oddParity: 12/56 operators
rotateRightBits: 8/16 operators
roundEvenPow2: 12/24 operators
midpointTowardFirst: 23/32 operators
isBetweenEitherOrder: 46/48 operators
mul5Sat: 23/30 operators
classifyAdd3: 39/52 operators
floatScaleThreeHalves: 36/60 operators
floatRoundEven: 42/65 operators
float_i2f: 32/40 operators
bitCount: 36/40 operators
bitReverse: 28/34 operators
All 19 functions passed operator checks.
```

```
yangxinyuan@localhost:~/ICS2026/Lab1$ ./btest
Score    Rating  Errors  Function
1         1       0      signMask
2         2       0      bitXor
3         3       0      negativePart
4         4       0      copyByteWithin
4         4       0      logicalShift
4         4       0      swapNibblePairs
4         4       0      secondLowestZeroBit
5         5       0      oddParity
5         5       0      rotateRightBits
5         5       0      roundEvenPow2
5         5       0      midpointTowardFirst
7         7       0      isBetweenEitherOrder
7         7       0      mul5Sat
7         7       0      classifyAdd3
7         7       0      floatScaleThreeHalves
10        10      0      floatRoundEven
10        10      0      float_i2f
10        10      0      bitCount
10        10      0      bitReverse
Total points: 110/110
```